prosaic

Alpha · 0.1.0-alpha.2

TYPESCRIPT SDK / QUICKSTART

Your first Prosaic integration.

Connect to your workspace, find an entity and read its bank transactions. Typed methods and automatic pagination handle the HTTP details.

> **You'll need:** Node.js 22+, a Node project and an API key from your workspace administrator. Keep keys on your server, outside source control.

01 Install

Install the alpha from npm and set your API key in your terminal.

```
yarn add prosaic-node@next
yarn add --dev tsx
export PROSAIC_API_KEY='YOUR_API_KEY'
```

02 Connect

Create `quickstart.mts`. Your first call returns the authenticated user.

Default API origin: https://app.prosaic.works

```
import Prosaic from 'prosaic-node';

const apiKey = process.env.PROSAIC_API_KEY;
if (!apiKey) throw new Error('Set PROSAIC_API_KEY');
const prosaic = new Prosaic(apiKey);
const { data: me } = await prosaic.me.retrieve();
console.log('Connected as', me.name);
```

03 Find an entity

List the books you can access. Append this example and run the file to find an entity ID.

```
const { data: entities } = await prosaic.entities.list();
console.table(
  entities.map(({ id, name }) => ({ id, name })),
);
```

04 Read transactions

Append this example. Use an ID from step 3 and choose your dates. Iteration fetches every page.

Use `await` instead of `for await` for a single page.

```
const entityId = 'REPLACE_WITH_ENTITY_ID';
for await (const txn of prosaic.transactions.list(
  entityId, {
    dateFrom: '2026-04-01',
    dateTo: '2026-04-30',
    pageSize: 100,
  },
)) {
  console.log(txn.id, txn.date, txn.amount);
}
```

Run your file. An empty list means no accessible records match. `yarn tsx quickstart.mts`

Connecting multiple customers?

Use OAuth with PKCE for delegated access. Ask Prosaic to register your app. Initialise the SDK with `{ accessToken }`; the package README covers consent and token refresh.

Handle errors

Catch `APIError` from `prosaic-node`; inspect `status`, `code` and `requestId`. Check your key for 401 and permissions for 403. Reads retry transient failures; writes are never automatically retried.

Keep financial data exact

Keep monetary values as returned; use Decimal.js for calculations. Send calendar dates as `YYYY-MM-DD`. Financial date filters follow NZ time. Most read responses wrap results in `data`.